Software Requirements Engineering

Requirements Engineering Process - Elicitation

Tibebe B.

Outline

- Overview
- Domain Understanding and Elicitation
- Requirements Elicitation Activities
- Requirements Elicitation Techniques
- Classifying Customers' Inputs

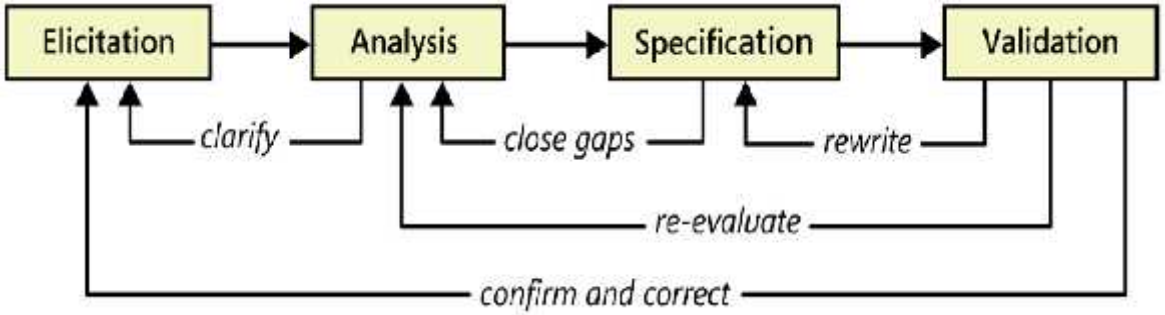
Objectives

- Understand requirement elicitation in the context of RE process
- List the objectives of requirement elicitation
- Describe the importance of domain understanding in requirement elicitation
- Describe requirement elicitation activities
- Explain the different requirement elicitation techniques
- Differentiate customers' inputs into requirements

Overview

RE Process

- Requirements development involves elicitation, analysis, specification, and validation
- These activities are not linear; they are interwoven and iterative



Requirement Elicitation - Objective

- It is the initial and main process of the requirements engineering phase.
- The process of obtaining a **comprehensive understanding** of stakeholder's requirements in the problem domain
- Usually involves interaction with stakeholders to obtain their real needs.
- It helps users to express their needs and expectations from the new system.
- High-quality requirements are obtained when the right people are chosen and are involved in elicitation process.

Domain Understanding & Requirement Elicitation

Domain Understanding & Requirement Elicitation

- The preliminary phase of the RE process, involves **a great deal of knowledge acquisition**

- Contextual knowledge under which the system-to-be will be elaborated
  - **Knowledge about the organization**: its structure, business objectives, policies, roles, responsibilities
  - **Knowledge about the domain** in which the problem world is rooted: concepts in the domain, objectives, regulations …
  - **Knowledge about system-as-is**: objectives, actors, resources, tasks, workflows and the problems raise in this context

Cont…

- Understand the system-as-is in context
- Identify problems and opportunities calling for a new system
- Identify the real needs of stakeholders
- Explore alternatives in which the new system could address those needs

Cont…

- The output of the elicitation phase is a **draft proposal report** describing the system-as-is, the organization, domain area, the problems, opportunities, and alternative ways of in which problems might be addressed

- Requirement elicitation and domain understanding techniques
  - **Artifact-driven**: rely on artifacts
  - **Stakeholder-driven**: rely on interaction with stakeholders

- Identify the **right stakeholders** for effective knowledge acquisition

Requirements Elicitation Activities

Activities

- Define product vision and scope
- Identify Stakeholder and their interest
- Identify actors
- Identify scenarios
- Identify and define use cases and user stories

Define Product Vision & Scope

- The **vision** and scope contains the product's **business requirements**.
- The vision gives all stakeholders a **common understanding** of the product's outcome
- Articulates the **overarching** goals of the software product.
- It serves as a motivational statement that outlines **what the product aims to achieve** and **why it is being developed**

- The **scope** defines the **boundary** between what's in and what's out for a specific release or iteration
- The scope details **how** the Product Vision will be realized.
- The vision should remain **relatively stable** throughout the project
  - But each planned release or iteration needs its own scope statement.

Identify Stakeholder and their interest

- Stakeholders should be selected based on their **roles, stakes, interests** and **type of knowledge** they contribute
- **Stakeholder cooperation** is essential for building shared understanding of the problem to be addressed by the system-to-be
- Criteria for **selecting** stakeholders
  - Relevant position in the organization
  - Role of making decision about the system-to-be
  - Level of domain expertise
  - Exposure to perceived problems
  - Influence in system acceptance
  - Personal objectives and conflicts of interest

Cont…

- Obstacles to effective knowledge acquisition from stakeholders
  - **Distributed and conflicting** knowledge sources: many different sources to consider; multiple stakeholders and large volumes of documents and data
  - **Difficult access to source**: key people are generally very busy
- Obstacles to good communication
  - **Tacit knowledge and hidden needs**: getting key information might be quite hard.
  - Knowledge is usually tacit, it is **implicit in the stakeholder's mind**
- Interacting with stakeholders
  - Communication skills
  - Knowledge reformulation

Identify Actors

- Identify all **potential system users** and **other systems** that might interact with it.
- Questions whose answers help in identifying actors:
  - Which user groups does the system support to do their work?
  - Which user groups execute the system's primary functions?
  - Which user groups execute the system's secondary functions?
    - E.g., maintain or administer the system
  - With which external systems does the system interact?
    - E.g., hardware or software

Identify Scenarios

- Scenarios are **representation of user's interaction** with the system.
  - It is a real-world example of how a system is used.
- Scenarios can be used once the initial requirements are collected.
- It includes whole **description of all the processes**
  - i.e. starting state, flow of events, concurrent activities, end state etc.
- They are useful when system needs to be described from the user's viewpoint.
- Writing a scenario **needs basic understanding of the tasks performed** by the system and the
- They are written in a **simple natural language.**
- Scenarios prove to be useful to validate the requirements and create test cases

Cont…

- Illustrates a typical **sequence of interactions** among system components that meets an implicit objective
- Widely used throughout the software lifecycle
- **Explaining how the system-as-is** is running through concrete examples of Realtime interaction sequences
- **Exploring how the system-to-be** should be running with hypothetical interaction sequences

Cont…

Questions for identifying scenarios:

- What tasks to the actors want the system to perform?
- What data does the actor need?
  - Who creates, modifies, removes that data?
- Which external changes affect the system's state?
  - How is the change communicated to the system? (what actor?)
  - Under what circumstances?

Scenario - Example

- **Scenario Title:** *Booking a Flight through a Travel App*
- **Description:** A frequent traveler, Jane, opens the travel app on her smartphone to book a flight from New York to London. She wants to travel next month and is looking for an evening flight. She logs in, searches for flights, compares prices, selects her preferred option, enters her personal details, chooses a seat, and pays using her credit card. She receives an email confirmation with her e-ticket.

Exercise:

Requirement Elicitation – Scenario

- **Background Situation:**

  You have been hired to help develop a **Library Management System** for a university. The system should allow students and faculty to search for books, borrow them, and return them. The librarian should be able to add, update, or remove books from the catalog.

- **Task:** Write a **scenario** (a short story or real-world situation) based on one of the following actions:
  - A student wants to borrow a book.
  - A librarian updates the book inventory.
  - A faculty member searches for a reference book.

Sample Answer (Solution)

- Robel, a second-year CS student, wants to borrow a book *"RE Techniques"* for his term project. He logs into the university library portal, types the book's title into the search bar, and finds several copies. He sees that one is available, clicks "Borrow," and selects the pickup location. He gets a notification that his book is ready for collection.

Identify and define Use Cases and user stories

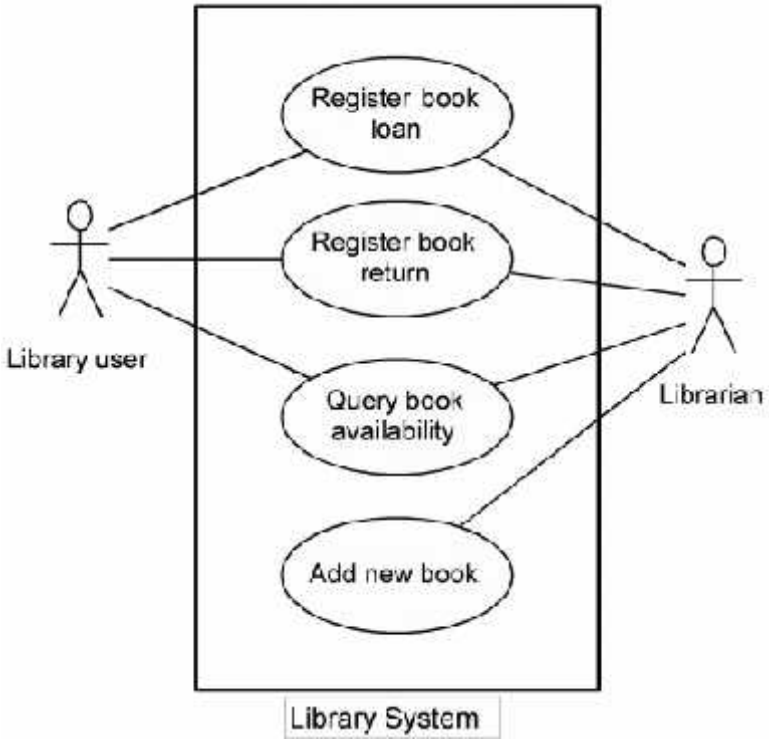
- A **use case** describes a **sequence of interactions** between a system and an actor that results in the actor being able to achieve some outcome of value.
- The names of use cases are always written in the form of a **verb followed by an object**.

- Use case usually are represented using
  - Use case diagrams (use cases, actors, boundary)
  - Use case description

User story

- A **user story** is a "short, simple description of a feature told from the perspective of the person who desires the new capability, usually a user"
- User stories can be written using the following template,
  - As a <type of user>, I want <some goal> so that <some reason>
  - Ex. As a traveler, I want to check in for a flight so that I can fly to Dubai

Use case diagram example



Cont…

TABLE 8-1 Sample use cases from various applications

Application	Sample use case
Chemical tracking system	Request a Chemical Print Material Safety Data Sheet Change a Chemical Request Check Status of an Order Generate Quarterly Chemical-Usage Reports
Airport check-in kiosk	Check in for a Flight Print Boarding Passes Change Seats Check Luggage Purchase an Upgrade
Accounting system	Create an Invoice Reconcile an Account Statement Enter a Credit Card Transaction Print Tax Forms for Vendors Search for a Specific Transaction
Online bookstore	Update Customer Profile Search for an Item Buy an Item Track a Shipped Package Cancel an Unshipped Order

Elements of a Use Case Description – Example

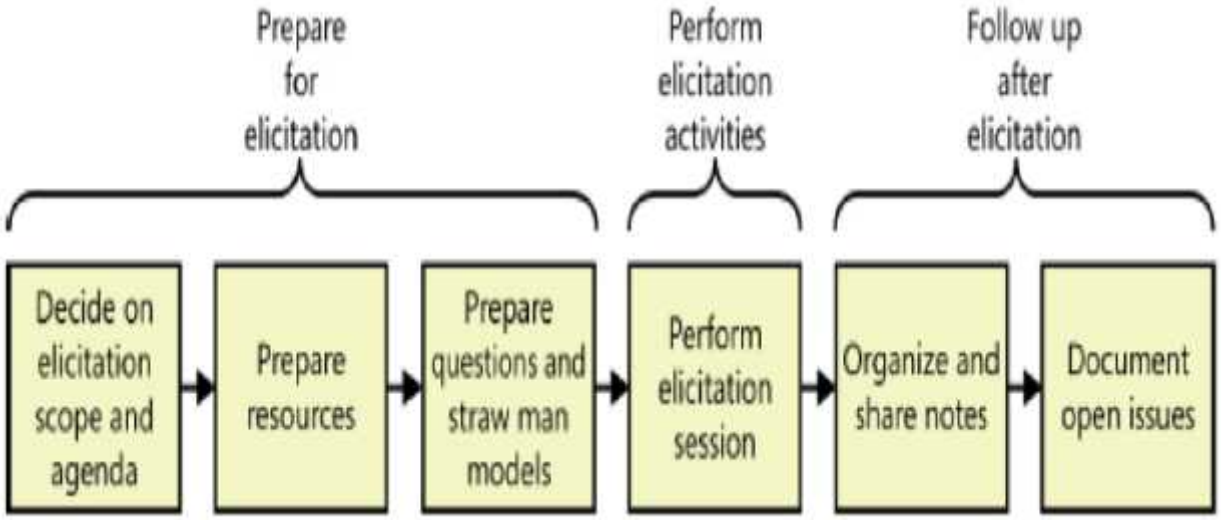
- Use case name Register for a semester
- Actor[s] Student, advisors, registrar, bank
- Entry conditions Previous semesters cumulative GPA is above 2.0
- Flow of events
  1. Student checks whether is able to register
  2. Registrar confirms if student is able to register
  3. Student settles registration and related fees
  4. Student submits payment slip to advisor
  4. Student receives registration slip from advisor
  5. Student receives registration approval from advisor
  6. Registrar registers
- Exit conditions Student is registered
- Special requirements Student is able to register on the notified dates only

Use cases and corresponding user stories

TABLE 8-2 Some sample use cases and corresponding user stories

Application	Sample use case	Corresponding user story
Chemical tracking system	Request a Chemical	As a chemist, I want to request a chemical so that I can perform experiments.
Airport check-in kiosk	Check in for a Flight	As a traveler, I want to check in for a flight so that I can fly to my destination.
Accounting system	Create an Invoice	As a small business owner, I want to create an invoice so that I can bill a customer.
Online bookstore	Update Customer Profile	As a customer, I want to update my customer profile so that future purchases are billed to a new credit card number.

A single requirements elicitation session



Requirements Elicitation Techniques

Requirements Elicitation Techniques

- Document Analysis
- Interview
- Questionnaires
- Observation
- Ethnography
- Prototyping
- Joint Application Development (JAD)
- Requirement Workshops
- Storyboards

Document Analysis

- Analyzing and gathering information from **existing documents** and other related information
- Used to **initiate** the requirements elicitation process

- This technique is used when an expert needs **to study domain information thoroughly**
- Analyzing documents such as **forms**, **design documents**, **templates**, and **manuals** of existing systems.
- This technique is adapted when the existing system needs to be replaced or enhanced.

Pros and Cons of Document Analysis

- Pros:
  - Helpful when stakeholders and users are not available.
  - Helps analysts get an understanding of the organization before meeting stakeholders.
  - Can be useful to frame questions for interviews.
  - Can be used for requirements **reuse**.
  - **Inexpensive** technique.
- Cons:
  - Time-consuming to find information from the huge amount of documentation.
  - Sometimes valid information may not be available i.e. **documents may be outdated**.
  - Periodic updating of documents is required.
  - Information might be incomplete.

Interview

- It is a conversational or verbal method that is effective for **idea sharing** and **expressing needs between analysts and stakeholders**
- Most employed technique
- It is considered an important technique to obtain and validate software requirements
- Interviews are of **3 types:**
  - **Structured:** formal with a set of predefined questions
  - **Semi-structured**: a combination of predefined and unplanned questions
  - **Unstructured**: informal interview containing unplanned questions

Pros and Cons of Interview

- Pros of Interviews:
  - Good for **complex topics**.
  - **Rich in information** and good in eliciting detailed requirements.
  - No answer falsification by the person being interviewed as he is being screened face-to-face.
  - The interviewer can analyze emotions and body language
  - Non-responsiveness remains low.
  - Provides an overview of the whole system.
- Cons of Interviews:
  - **Small number** of people involved.
  - Cost can be a constraint.
  - Quality of data gathered depends on the skills of the interviewer.
  - They are effortful and **time-consuming**.

Questionnaires

- It is considered the **cheapest** way of eliciting requirements
- This technique is used to collect requirements from a **larger group** of population distributed over a large geographical area
- Questionnaires must be clear, well-defined and precise besides including the domain knowledge

Pros and Cons of Questionnaires

- Pros:
  - Reach large number of people within a short time.
  - Useful when same question is asked to large number of people.
  - Economical.
- Cons:
  - Cannot get further clarification regarding the problem what analyst actually wants from the user.
  - Questions can be **misinterpreted**.
  - Sometimes useful feedback isn't received.
  - Sometimes question ambiguities may arise.
  - Used for general purpose software.

Observation

- The requirements engineer **observes the user's environment** without interfering in their work
- Used when the customer is not able to explain what they want to see in the system, how they work, and when some ongoing processes are to be monitored
- Usually support by a checklist

- Used with other elicitation techniques
- Observation can be done actively or passively.
  - **Passive observation** is when the analyst doesn't interact with the user while he is observing.
  - **Active observation** is when the user is interrupted for questions during observation

Pros and Cons of Observation

- Pros:
  - **Authentic** and **reliable** as analysts observe the environment.
  - Can be useful to confirm and validate requirements collected through other methods.
  - It is inexpensive.
  - Gives idea about how users will interact with the system.
  - Helpful in work measurements i.e. how long particular task takes to be done.
- Cons:
  - All the requirements cannot be checked in just a single session; multiple sessions may be required.
  - Users can behave indifferently while they are interrupted for asking questions in active observation.
  - In passive observation, it is difficult for analyst to make out why some decisions are made.
  - It is time consuming.

Ethnography

- Observe the activities of people from different communities for some time in **detail** while collecting the requirements needed
- Observe a particular workplace and actors and relationships between them

- Analyst involves himself completely in the organizational environment to understand its socio-organizational requirements
- Good for dealing with contextual problems like **usability** and detailed observation of collaborative workplaces
- Usually used with other techniques
- Applies for business processes involving multiple stakeholders who may not realize what the others are doing or have difficulties explaining it

- The major advantage is the **ability to reveal tacit knowledge**

Pros and Cons of Ethnography

- Pros:
  - Helps to discover certain features of a workplace in a shorter time.
  - Helps understand how people work in an organization and how they interact with each other.
  - Doesn't need much resources to be effective.
  - Helps reveal critical events not observed by any other technique.
  - Useful in validating requirements.
- Cons:
  - There is no detailed guide on how to perform ethnographic technique effectively and therefore, it all depends on the skills of the person performing it, the ethnographer.
  - Fails to produce desirable results due to diverse population.
  - Focuses mainly on end-users.
  - Sometimes it can be time consuming.
  - Different backgrounds of users and ethnographers can result in misunderstanding problems between them.

Prototyping

- An iterative process in which **dummy version of the product** is launched to obtain user requirements and is refined over and over again according to the user feedback
- Used where the users have **no/little knowledge about their requirements** and where early response is required from the stakeholders
- Can be used with other techniques such as JAD
- Gives an idea to the user about what he actually wants from the system by providing him the basic interface design.
- **Prototypes** are mainly designed in systems which have more user interactions and less internal functionality

Prototypes

- Stakeholders usually find difficulties to project textual system descriptions onto future working conditions; we might show them reduced sketch of the product to help visualize the system-to-be
- A ***software prototype*** **is a quick implementation** of some aspects of the system-to-be.
- Aimed at **getting early feedback** from stakeholders
- Focused on requirements that are unclear, hard to formulate or hard to understand
- Two types
  - **Functional prototype** shows aspects related to software functionalities
  - **User interface prototype** shows static and dynamic aspects of user-software interaction: formats for data entry and output display, and dialogue patterns

Pros and Cons of Prototyping

- Pros
  - User involvement during development process.
  - Allows early user feedback for requirements refinement.
  - Saves development time and cost.
  - Users and analysts get better understanding of the system.
- Cons
  - The disadvantage is that when users get used to particular kind of system, they often resist changes.
  - Effort and cost estimation may get high as calculated earlier.
  - For complex systems, it can be time consuming.

Joint Application Development

- Are basically **collaborative workshops** that last for 4-5 days
- It helps to gather lot of information in a shorter period of time
- Participants share their opinions on what must be done and what needs to be changed
- Sometimes **prototype may** also be the end product of the JAD session to refine the system.
- Visual Aids like monitors and GUI is also used to make the session interactive
- Prototype may also be the end product of the JAD session

JAD Pros and Cons

- Pros:
  - **Decreased time and cost of** requirements elicitation.
  - Accelerates design of the system
  - New and rapid idea generation leading to creative outputs.
  - Promotes user feedback.
  - More user satisfaction.
  - Good communication between stakeholders, analysts and other professionals.
  - Visual aids and case tools used make the session interactive.
- Cons:
  - If not properly planned can lead to wastage of time and resources.
  - Requires trained facilitators.
  - Requires lots of planning and effort.
  - It is an expensive technique.

Requirement Workshops

- It is a **structured meeting** where selected stakeholders come together to discuss, refine and validate requirements after multiple sessions.
- These requirements are **unchangeable** as they are collected after multiple sessions.
- Expertise of facilitator and knowledge of stakeholders is must for its success.
- Participants discuss and identify requirements then they are reviewed and assign priorities.
- It has some similarity to JAD

Requirement Workshops Pros and Cons

- Pros:
  - Collect requirements in a relatively shorter time.
  - Cost is lower than performing interview sessions.
  - Feedback is quick.
  - Helps to develop trust and understanding between analysts and stakeholders.
  - Participants work together to come up with a particular set of requirements.
- Cons:
  - Due to busy schedule of stakeholders and experts it is difficult to get them on the same table at the same time.
  - Too many participants may slow down the process.
  - Few number of participants may lead to requirements that are incomplete and doesn't represent proper set of user requirements.

Storyboards

- Use of narratives proves effective in the early stages of the RE process to elicit or validate useful information
- Such narratives are called storyboards
  - **Passive storyboard**: stakeholders are told the story. The storyboard is used for explanation or validation
  - **Active storyboard**: stakeholders contribute to the story. Used for joint exploration
- Storyboards break down a user's interaction with a system into a sequence of steps, making it easier to visualize and understand the flow of events.

Example od story boards in a Social Media App

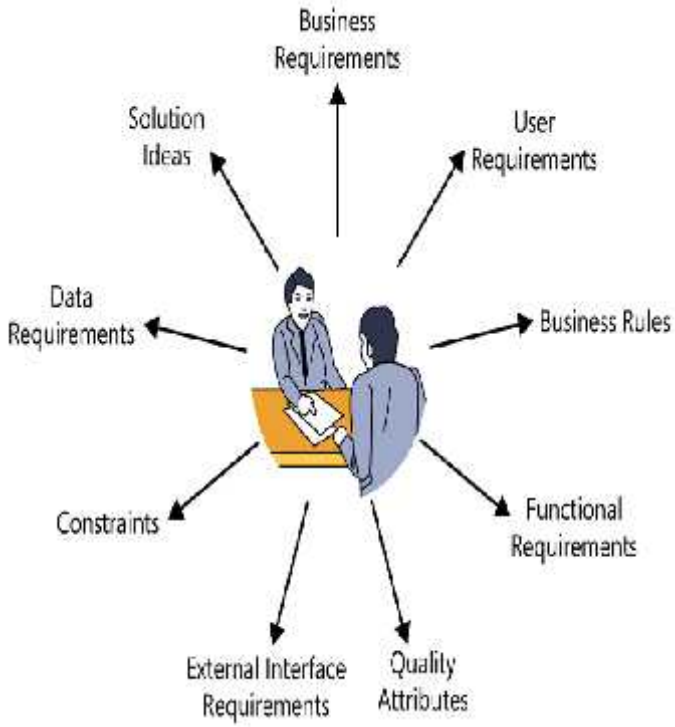
- **Scene 1:** A user opens the app on their phone.
- **Scene 2:** The user sees their timeline with posts from friends.
- **Scene 3:** The user clicks the "add post" button.
- **Scene 4:** The user types their message.
- **Scene 5:** The user clicks "post".
- **Scene 6:** The user sees their new post on their timeline, with their profile picture and timestamp.
- **Scene 7:** Another user sees the new post on their timeline and clicks to like it.

Suggested RE techniques by project characteristics

	Interviews	Workshops	Focus groups	Observations	Questionnaires	System interface analysis	User interface analysis	Document analysis
Mass-market software	x		x		x			
Internal corporate software	x	x	x	x		x		x
Replacing existing system	x	x		x		x	x	x
Enhancing existing system	x	x				x	x	x
New application	x	x				x		
Packaged software implementation	x	x		x		x		x
Embedded systems	x	x				x		x
Geographically distributed stakeholders	x	x			x			

Classifying Customers' Input

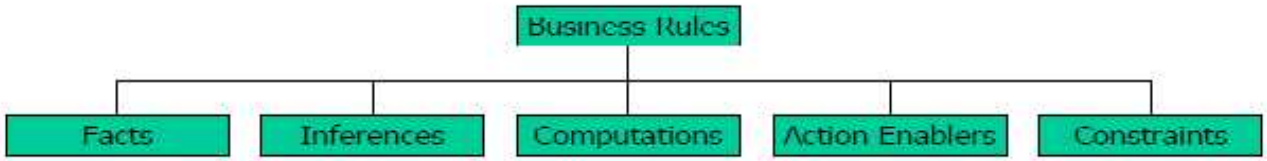
- Customers don't present a succinct, complete, and well-organized list of their needs
- REs must classify these requirements into various categories
- Understanding different user categories is thus basic
  - Some might tell you high level requirements
  - Others more specific business rules
  - Still other detailed tasks that they want the system to hep them



Classifying Customers' Input

- **Business Requirements**: Anything that describes the financial, marketplace, or other business benefit
  - Statements about the value that users of the software will receive
  - "Increase market share in region X by Y percent within Z months."
- **User Requirements**: General statements of user goals or business tasks that users need to perform
  - Most typically represented as use cases, scenarios, or user stories
  - "I need to <do something>"
- **Business Rules**: When a customer says that only certain users can perform an activity under specific conditions
  - "If <some condition is true>, then <something happens>,"
  - "A new client must pay 30 percent of the estimated consulting fee and travel expenses in advance."

More details on Business Rules



- ***Fact*** – **immutable truth about data entities and their attributes.**

  E.g. "Sales tax is not computed on shipping charges".

- ***Inference*** – **establishes some fact based on the truth of certain conditions.**

E.g. "Explosive chemicals are considered expired one year after its manufacture date".

- ***Computation*** – **a rule involving a formula or an algorithm.**

  E.g. "The unit price is reduced by 10% for orders of 6 to 10 units, and by 20% for orders of more than 10".

- ***Action enabler*** - **something that is to be done under specified conditions**.

E.g. "When a chemical container expires, the person who possesses it must be notified".

- ***Constraint*** – **a restriction on possible actions**.

E.g. "A borrower under 18 must have a parent or a legal guardian as cosigner on a library loan".

Classifying Customers' Input

- **Functional Requirements**: Describe observable behaviors the system will exhibit under certain conditions and the actions users take
- **Quality Attributes**: Describe how well the system does something
  - Focus on words that describe desirable system characteristics: fast, easy, user-friendly, reliable, secure, etc.
- **External Interface Requirements**: Describe the connections between your system and the rest of the universe.
  - "Must read signals from . . . ," "Must send messages to . . . ," "Must be able to read files in <format>,"

Classifying Customers' Input

- **Constraints**: Legitimately restrict the options available to the developer
  - Constraints such as size, weight, and interface connections
  - "Must be written in <a specific programming language>," "Cannot exceed <some limit>,"
- **Data Requirements**: Describe the format, data type, allowed values, or default value for a data element; the composition of a complex business data structure; or a report to be generated
- **Solution Ideas**: describes a specific way to interact with the system to perform some action
  - Many "requirements" from users are really solution ideas
  - Needs to probe below the surface of a solution idea to get to the real requirement

Some cautions about elicitation

- Good way of conducting elicitation discussions comes with experience and builds on training in interviewing, group facilitation, conflict resolution, and similar activities.
- However, a few cautions will decrease the learning curve.
  - **Balance stakeholder representation**
  - **Define scope appropriately**
  - **Avoid the requirements-versus-design argument**

Cont…

- If customers request functionality similar to that already present in an existing product, see whether the requirements can be reused
- Reusing or Adapting the existing components.
  - Multiple projects will reuse those requirements that comply with an organization's business rules.
- Ex.
  - security requirements
  - requirements that conform to government regulations

Exercise

- Identify and name an application/system (hypothetical or real) and Identify two use cases. Then associate use case with user stories. One user story for one use case.

Readings

- Sommerville, Ian (2017) Software Engineering, Tenth edition,
  - Chapter 4, Section 4.3
- Elizabeth Hull, Ken Jackson and Jeremy Dick (2010)Requirements Engineering
  - Chapter 5
- Wiegers, Karl E. (2013). Software Requirements, third Edition Microsoft Press
  - Chapter 7

End of Chapter 2